

수업 07 · 전체 과정 12/20일차

OpenCV + 웹캠 실습

2026-10-22 (목) · 10:00~17:00

50분 수업 · 10분 휴식 · 중식 11:50~13:00

이미지의 픽셀, 좌표, 색을 읽고 밝은 영역을 찾습니다. 파일 예제로 연습한 뒤 웹캠의 열기와 종료 흐름을 살펴봅니다.

교안 읽기	직접 해보기	결과 확인하기
개념과 완성 예제를 먼저 봅니다.	필수 활동을 열어 작성·수정합니다.	예상과 실제를 비교하고 설명합니다.

처음 배우는 사람을 위한 수업

파일과 폴더, 저장과 실행부터 설명합니다. 설치·계정 문제가 있으면 제공 예제의 수동 추적이나 강사 시연으로 같은 개념을 확인하고, 실제 실행 여부를 구분해 기록합니다.

웹 강의실 열기 · 화면의 링크와 내려받기 자료를 함께 사용하세요.

PDF 제작일 2026-10-05 · 원본 웹 내용 + 50분 교시 운영·설명 보완

이 교안의 순서

오늘 준비할 것	3
오늘의 6교시	4
1교시 · 이미지와 프레임의 기초	5
2교시 · 읽기·좌표·변환	10
3교시 · 밝기와 연결 영역	15
4교시 · 조건 비교와 검출 한계	22
5교시 · 프레임 반복과 웹캠	27
6교시 · 새 요구와 결과 검수	33
용어와 코드 기호 찾아보기	38
오늘 배운 내용 확인하기	39
파일과 다음 수업	42

오늘 준비할 것

2026-10-22 · 처음 시작

기본 경로는 파일 이미지입니다. 개인 실행은 Python 3 설치 후 터미널에서 `python -m pip install opencv-python==4.12.0.88`을 사용할 수 있는 환경이 필요합니다. 설치 권한이 없으면 강사 시연·제공 출력 비교로 진행합니다. 실제 웹캠 스크립트는 강사가 장치·권한·촬영 범위를 확인한 뒤에만 선택해서 실행합니다.

파일과 실행

`examples/course/data`는 읽을 입력, `code`는 완성 시연 코드, `work`는 나의 작성·수정 파일입니다. ZIP을 풀고 폴더 구조를 유지합니다. 터미널은 명령을 실행하는 창이며, 에디터는 파일 내용을 쓰고 저장하는 창입니다. 저장만으로 코드가 실행되지는 않습니다.

1. 용어에서 낯선 말을 확인합니다.
2. 교안의 개념과 예를 읽습니다.
3. 활동지의 A01부터 입력을 열고 work 양식에 기록합니다.
4. 코드 실행은 code/line-guide.md의 명령과 조건을 따릅니다.

확인할 것: 기본 설정에서 검출 수가 2개이고 임계값을 60으로 바꾸면 3개가 되는 이유를 설명해 보세요. 파일 읽기 실패와 검출 결과가 0개인 경우를 구별하고, release로 카메라 자원을 해제하는 이유도 설명합니다.

실제 웹캠과 화면 창의 동작은 강의실에서 별도로 확인해야 합니다. 제공 파일을 사용하는 실습과 실제 장치의 결과를 구별해 기록하세요.

수업 전, 이 세 가지부터 확인하세요

앞선 수업을 기억하지 못해도 아래 질문과 풀이부터 다시 읽으면 됩니다. 먼저 자기 말로 답한 뒤 풀이와 비교하세요.

픽셀

질문: 이미지 폭 100과 높이 60이면 (80,70)은 안쪽인가요?

풀이: $x=80$ 은 폭 안쪽이지만 $y=70$ 은 높이 범위를 벗어납니다. 좌표는 보통 0부터 세며 배열 접근의 행·열 순서를 확인합니다.

검출

질문: 밝은 영역 두 개를 찾으면 물체 두 개라고 확정하나요?

풀이: 그 설정에서 조건을 만족하는 영역 두 개를 찾은 결과입니다. 반사·그림자·불은 영역 때문에 실제 물체와 다를 수 있습니다.

비교

질문: 밝기 기준과 최소 면적을 함께 바꾸면 이유를 알 수 있나요?

풀이: 조건을 하나만 바꾸고 나머지를 고정해야 어떤 변화가 결과에 영향을 주었는지 비교하기 쉽습니다.

오늘의 수업과 복습을 나누는 방법

수업은 10:0010:50, 11:0011:50, 13:0013:50, 14:0014:50, 15:0015:50, 16:0016:50입니다. 중식은 11:5013:00이며 수업 사이에는 10분 쉽니다. 마지막 16:5017:00은 휴식과 저장·퇴실 준비로 배치했습니다.

교시	수업에서 먼저 확인할 활동	추가·복습
1	A01, A02, A03	새 입력으로 한 번 더 확인
2	A04, A05, A06	새 입력으로 한 번 더 확인
3	A07, A08, A09	새 입력으로 한 번 더 확인
4	A10, A11, A12	새 입력으로 한 번 더 확인
5	A13, A14, A15	새 입력으로 한 번 더 확인
6	A16, A17, A18	새 입력으로 한 번 더 확인

교안의 쉬운 사례와 단계별 풀이를 먼저 읽고 필수 결과를 직접 만들어 봅니다. 새 독립 문제는 결과를 확인한 뒤 복습하거나 도전합니다. 모든 페이지를 같은 시간에 끝내라는 뜻은 아닙니다. 설치가 막혔을 때는 제공 출력과 수동 추적으로 개념을 확인하고, 직접 실행했는지는 따로 적습니다.

오늘의 6교시

교시·시간	학습 초점	필수 활동
1교시 10:00~10:50	픽셀·좌표·프레임	A01, A02, A03
2교시 11:00~11:50	읽기·관심 영역·회색조	A04, A05, A06
3교시 13:00~13:50	임계값과 면적 조건	A07, A08, A09
4교시 14:00~14:50	한 조건 비교와 실패	A10, A11, A12
5교시 15:00~15:50	프레임 반복과 장치 종료	A13, A14, A15
6교시 16:00~16:50	요구와 결과 묶음	A16, A17, A18

설명·시연·함께 수행·개별 수행·폴이를 합쳐 교시당 50분입니다. 추가 활동까지 모두 같은 시간에 끝내라는 뜻은 아닙니다. 마지막 10분은 휴식·종료 준비로 배치합니다. 실제 학습자 소요시간은 수업에서 측정해 조정합니다.

컴퓨터는 사진을 의미가 붙은 사물 목록으로 바로 읽지 않습니다. 이번 수업은 배열의 위치와 색 값에서 시작해, 어떤 조건의 영역을 남겼는지 설명할 수 있는 작은 영상 처리를 만듭니다. 규칙 기반 영상 처리와 학습한 AI 모델의 분류를 혼동하지 않습니다.

수업의 출발점

10/8의 데이터 구조와 10/14~15의 변수·조건·반복을 복습합니다. 배열은 줄과 칸으로 값을 담는 방식이며, OpenCV는 이 이미지 읽기와 변환을 제공하는 라이브러리입니다. import는 그 도구를 코드에서 쓰도록 가져오는 문장이고 설치와 실행은 별도 단계입니다. 파일 입력을 기본으로 사용하며 실제 카메라는 운영 조건이 갖춰진 경우의 선택 경로로 둡니다.

읽는 방법

개념의 뜻을 먼저 읽고 가상 입력에서 근거를 찾습니다. 최신 운영은 50분 수업 × 6교시이며 순수 수업 300분, 중식 11:50~13:00입니다. 기존 활동은 한 교시의 개념 보충·시연·필수 적용·폴이를 위한 자료 묶음입니다. 복습·도전 문제는 진도에 따라 선택합니다. 모든 활동을 50분 안에 모두 완수한다고 가정하지 않습니다. 실제 학생 수행시간은 아직 측정하지 않았습니다.

1교시 · 이미지와 프레임의 기초

10:00~10:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 폴이·확인 7분

이번 교시에서 확인할 것. 픽셀·좌표·프레임

먼저 할 활동: A01, A02, A03

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A01 · 이미지는 작은 값들의 격자

1교시 필수 개념 보충 · 좌표를 알아야 검사 위치를 정할 수 있다

실무 상황과 원리

포장 검사 담당자가 사진 오른쪽 위의 인쇄만 확인하려고 합니다. 사진 파일을 보았다는 사실과 컴퓨터가 그 위치를 읽었다는 사실은 다릅니다. 파일은 저장된 자료이고, 배열은 읽은 뒤 메모리에서 값이 놓인 표입니다. 격자의 한 칸인 픽셀에 번호를 붙이면 위치를 정확히 전달할 수 있습니다. 왼쪽 위를 출발점으로 가로 번호 x , 세로 번호 y 가 각각 0부터 시작합니다.

OpenCV 컬러 배열의 shape는 높이, 너비, 채널 수 순서입니다. 높이 3, 너비 4, 채널 3이면 (3,4,3)이며 위치는 12곳, 색 성분은 총 36개입니다. image[y,x]에서 첫 번호는 행이므로 image[1,2]는 $x=2$, $y=1$ 입니다. BGR 값 [0,0,255]는 빨강 성분만 큰 값입니다. RGB 약속을 쓰는 다른 도구로 가져갈 때는 채널 순서를 확인해야 합니다.

완성 예를 종이에 추적합니다. 입력은 shape (3,4,3), 질문은 오른쪽 끝 아래 칸입니다. 처리에서 x의 마지막 번호는 $4-1=3$, y의 마지막 번호는 $3-1=2$ 로 계산합니다. 출력 위치는 image[2,3]입니다. 번호 4와 3을 그대로 쓰면 범위를 벗어 납니다. 해상도만으로 실제 물체가 몇 mm인지 계산하지 못하며 촬영 거리와 기준 길이가 필요합니다.

새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 높이 5, 너비 8인 새 사진의 왼쪽 아래 픽셀과 전체 위치 수를 직접 작성하세요. 채널이 3일 때 색 성분 수도 별도로 쓰세요.

정답과 오답의 이유

정답은 image[4,0], 위치 40곳, 색 성분 120개입니다. image[0,4]는 첫 행의 다섯 번째 위치라서 아래 칸이 아닙니다. 120을 물체 개수로 쓰면 색 성분과 검사 대상을 혼동한 것입니다.

확인 질문과 답

1. shape의 첫 값은? → 높이입니다.
2. $x=3, y=2$ 의 배열 표기는? → image[2,3]입니다.
3. BGR의 마지막 값은? → 빨강 성분입니다.

픽셀은 이미지에서 한 위치의 색이나 밝기를 담는 최소 단위입니다. 모눈종이의 칸과 비슷하지만 실제 물체의 고정 길이를 뜻하지는 않습니다.

값의 배열을 이해하면 색이 바뀌거나 좌표가 어긋난 이유를 설명할 수 있습니다. 이미지 파일의 이름만으로 내용과 크기를 알 수 없습니다.

입력에서 살펴볼 예

4×3 회색조 예 0 0 255 255 0 80 80 255 0 0 0 0

가로 4칸, 세로 3칸입니다. 0은 어둡고 255는 밝습니다. 이 예의 배열은 높이 3, 너비 4입니다.

구분해야 할 반례

픽셀 한 칸이 현실의 1mm라는 뜻은 아닙니다. 촬영 거리와 렌즈, 해상도에 따라 대응 길이가 달라집니다.

스스로 설명할 질문: 가로·세로와 실제 길이를 왜 구분해야 하나요?

활동 연결: [입력](#) · [기록 양식](#)

A01 · 따라 하기와 작성칸

A01 · 이미지는 작은 값들의 격자

목적 값의 배열을 이해하면 색이 바뀌거나 좌표가 어긋난 이유를 설명할 수 있습니다. 이미지 파일의 이름만으로 내용과 크기를 알 수 없습니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A01_input.md](#)

작업: [A01_worksheet.md](#)

단계

1. 격자에서 밝은 칸 세 개를 표시합니다.
2. 높이와 너비를 서로 다른 칸에 씁니다.
3. 한 픽셀의 실제 길이를 알기 위해 필요한 정보를 적습니다.

예상 결과 격자 읽기에 예상과 관찰 근거를 남깁니다.

확인 방법 가로·세로와 실제 길이를 왜 구분해야 하나요?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A02 · 좌표와 BGR 채널

좌표는 위치를 가리키는 번호이고 채널은 한 픽셀의 색 성분입니다. 이 수업의 OpenCV 컬러 입력은 파랑·초록·빨강 순서인 BGR입니다.

배열은 `image[y,x]`로 읽으므로 화면의 `x,y` 설명과 순서를 구분해야 합니다. 채널 순서를 바꾸면 빨강과 파랑의 해석이 바뀝니다.

입력에서 살펴볼 예

좌표: `x=2, y=1` 픽셀: `[0, 0, 255]` 컬러 shape: `(240,320,3)`

`image[1,2]`에서 BGR 값 `[0,0,255]`는 빨강입니다. shape는 높이 240, 너비 320, 채널 3입니다.

구분해야 할 반례

`[0,0,255]`를 모든 도구에서 파랑으로 읽지는 않습니다. RGB와 BGR은 채널 약속이 다릅니다.

스스로 설명할 질문: `image[40,30]`은 어느 `x,y`를 가리키나요?

활동 연결: [입력](#) · [기록 양식](#)

A02 · 따라 하기와 작성칸

A02 · 좌표와 BGR 채널

목적 배열은 image[y,x]로 읽으므로 화면의 x,y 설명과 순서를 구분해야 합니다. 채널 순서를 바꾸면 빨강과 파랑의 해석이 바뀝니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A02_input.md](#)

작업: [A02_worksheet.md](#)

단계

1. x,y와 배열 인덱스를 대응합니다.
2. BGR 세 값에서 빨강 성분을 표시합니다.
3. shape 세 숫자의 뜻을 씁니다.

예상 결과 좌표 대응에 예상과 관찰 근거를 남깁니다.

확인 방법 image[40,30]은 어느 x,y를 가리키나요?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A03 · 파일과 프레임의 차이

프레임은 영상에서 한 시점의 이미지입니다. 책의 한 장을 차례로 넘기는 것처럼 여러 프레임을 시간 순서로 보면 움직임을 볼 수 있습니다.

파일 입력은 같은 장면을 반복해서 시험하기 쉽습니다. 웹캠 입력은 매번 장면과 밝기가 달라질 수 있으므로 처리 규칙과 수집 조건을 따로 확인합니다.

입력에서 살펴볼 예

parts.png: 한 장면 empty.png: 물체 없는 장면 holdout.png: 새 배치 장면

세 파일을 순서대로 읽어도 실제 카메라 속도 검증은 아닙니다. 실시간 영상은 장치·시간·처리 지연을 추가로 확인합니다.

구분해야 할 반례

프레임 3장을 읽었다고 고유 물체 3개를 찾았다는 뜻은 아닙니다. 같은 물체가 여러 프레임에 반복될 수 있습니다.

스스로 설명할 질문: 파일 재생과 실제 웹캠 검증은 무엇이 다른가요?

활동 연결: [입력](#) · [기록 양식](#)

A03 · 따라 하기와 작성칸

A03 · 파일과 프레임의 차이

목적 파일 입력은 같은 장면을 반복해서 시험하기 쉽습니다. 웹캠 입력은 매번 장면과 밝기가 달라질 수 있으므로 처리 규칙과 수집 조건을 따로 확인합니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A03_input.md](#) · [parts.png](#) · [empty.png](#) · [holdout.png](#)

작업: [A03_worksheet.md](#)

단계

1. 세 입력 파일의 역할을 적습니다.
2. 파일 반복 시험의 장점을 찾습니다.
3. 실제 웹캠에서 추가로 확인할 조건을 씁니다.

예상 결과 입력 종류에 예상과 관찰 근거를 남깁니다.

확인 방법 파일 재생과 실제 웹캠 검증은 무엇이 다른가요?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

2교시 · 읽기·좌표·변환

11:00~11:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 읽기·관심 영역·회색조

먼저 할 활동: A04, A05, A06

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A04 · 이미지 파일 읽기와 실패

2교시 필수 개념 보충 · 잘라내기와 밝기 변환이 바꾸는 정보

실무 상황과 원리

작업대 사진에는 부품과 함께 창문 반사가 들어 있습니다. 검사자는 작업대 가운데만 보고 싶습니다. 관심 영역은 전체 사진에서 필요한 작은 구역이며 ROI라고 부릅니다. 잘라내기는 바깥 픽셀을 제외하고, 크기 변경은 새 격자로 값을 다시 배치합니다. 둘을 같은 일이라고 생각하면 검사 위치와 면적 기준이 틀어집니다.

`image[20:80,30:130]`은 행 20부터 79, 열 30부터 129까지 가져옵니다. 콜론 뒤의 끝 번호는 포함하지 않습니다. 따라서 높이는 $80-20=60$, 너비는 $130-30=100$ 입니다. 잘린 사진의 (0,0)은 원본의 (30,20)입니다. 원본 좌표가 필요하다면 잘린 x에 30, y에 20을 더합니다. 가로와 세로를 각각 절반으로 줄이면 같은 물체의 면적은 대략 4분의 1이 됩니다. 정확한 경계 값은 다시 만든 픽셀과 윤곽선에 따라 달라집니다.

회색조는 여러 색 성분을 밝기 하나로 바꿉니다. 세 성분의 단순 평균이 아니라 변환 규칙을 사용합니다. 예제의 순수 빨강 `BGR [0,0,255]`는 대략 76, 초록 `[0,255,0]`은 150, 파랑 `[255,0,0]`은 29입니다. 밝은 부품을 찾기에는 편하지만 같은 밝기의 서로 다른 색은 구분하기 어렵습니다. 완성 흐름은 원본 읽기 → 필요한 영역 자르기 → 회색조 변환 → 높이와 너비 기록입니다. 회색조 결과의 shape는 (60,100)이며 채널 3이 사라집니다.

새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: `image[10:50,20:90]`을 새로 해석하고, 잘린 좌표 $x=4, y=7$ 의 원본 좌표를 계산하세요. 빨간 라벨과 녹색 라벨을 구별할 때 회색조만으로 충분한지도 판단하세요.

정답과 오답의 이유

높이 40, 너비 70, 원본 $x=24, y=17$ 입니다. 색이 업무 판단의 핵심이면 원본 색 정보를 보존해야 합니다. 끝 번호를 포함해 41×71 로 쓰거나 모든 색 차이가 밝기 차이로 유지된다고 쓰면 오답입니다.

확인 질문과 답

1. 끝 번호 50은 포함되나요? → 아닙니다.
2. 잘린 좌표의 출발점은? → 잘라낸 영역의 왼쪽 위입니다.
3. 반으로 축소한 면적 기준은? → 기존 기준을 그대로 확정하지 말고 재검증합니다.

imread는 파일을 이미지 배열로 읽는 함수입니다. 성공한 배열과 실패한 None을 구분한 뒤 다음 처리를 해야 합니다.

읽지 못한 파일을 회색조로 바꾸려고 하면 원인과 멀리 떨어진 오류가 날 수 있습니다. 입구에서 실패를 확인하면 경로·파일 이름을 먼저 복구할 수 있습니다.

입력에서 살펴볼 예

01_inspect.py 전체 data/parts.png 예상 shape (240,320,3)

코드가 파일을 찾고 배열 크기와 픽셀 값을 출력합니다. 회색조 결과는 work/gray.png에 저장합니다.

구분해야 할 반례

저장만 누르면 Python 코드가 실행되는 것은 아닙니다. 터미널에서 제공 명령을 실행하고 결과 파일 시각과 내용을 확인합니다.

스스로 설명할 질문: None 확인을 색 변환보다 먼저 하는 이유는 무엇인가요?

활동 연결: [입력](#) · [기록 양식](#)

01_inspect.py 전체

```
import cv2

from pathlib import Path

root = Path(__file__).resolve().parent.parent
source = root / "data" / "parts.png"
image = cv2.imread(str(source))

if image is None:
    raise ValueError("이미지를 읽지 못했습니다: " + str(source))

print("shape:", image.shape)
print("BGR:", image[40, 30].tolist())

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
saved = cv2.imwrite(str(root / "work" / "gray.png"), gray)

if not saved:
    raise OSError("출력 파일을 저장하지 못했습니다")

print("gray:", gray.shape)
```

실행 조건: Python 3와 OpenCV가 설치된 PC. 카메라나 GUI 창을 열지 않는 파일 입력 예제입니다.

실행 폴더: examples/course

명령: `python -X utf8 code/01_inspect.py`

결과:

```
shape: (240, 320, 3)
BGR: [220, 220, 220]
gray: (240, 320)
```

각 줄의 뜻은 [코드 해설](#)에 있습니다.

작은 수정: parts.png를 empty.png로 바꾸면 크기는 같고 지정 픽셀 값은 [0, 0, 0]입니다.

오류 복구: ModuleNotFoundError이면 설치 환경을 확인합니다. 읽기 오류면 압축을 풀고 data 폴더와 파일 철자를 확인합니다.

A04 · 따라 하기와 작성칸

A04 · 이미지 파일 읽기와 실패

목적 읽지 못한 파일을 회색조로 바꾸려고 하면 원인과 멀리 떨어진 오류가 날 수 있습니다. 입구에서 실패를 확인하면 경로·파일 이름을 먼저 복구할 수 있습니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A04_input.md](#) · [01_inspect.py](#) · [parts.png](#)

작업: [A04_worksheet.md](#)

단계

1. 실행 폴더와 입력 경로를 찾습니다.
2. 코드 각 줄을 읽고 크기·픽셀 값을 예상합니다.
3. 실행 또는 제공 출력과 비교하고 gray.png를 확인합니다.

예상 결과 실행 경로에 예상과 관찰 근거를 남깁니다.

확인 방법 None 확인을 색 변환보다 먼저 하는 이유는 무엇인가요?

오류 해결 cv2 설치 오류와 입력 경로 오류를 구분합니다. 제공 출력 비교로 계속하고 실제 실행 여부를 기록합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A05 · 관심 영역과 크기

관심 영역 ROI는 이미지에서 이번 문제에 필요한 부분입니다. 책에서 필요한 문단을 고르는 것처럼 분석할 범위를 먼저 정합니다.

배경까지 처리하면 관계없는 밝은 물체도 함께 잡힐 수 있습니다. 잘라내기와 크기 변경은 목적과 좌표를 바꾸므로 원본 크기를 기록해야 합니다.

입력에서 살펴볼 예

원본 shape (240,320,3) ROI: image[20:80,10:70] resize 목표: (160,120)

슬라이스 끝 번호는 포함하지 않아 ROI는 60×60입니다. resize의 크기 인자는 너비 160, 높이 120입니다.

구분해야 할 반례

자르기는 범위를 버리고 크기 변경은 전체를 새 격자로 만듭니다. 둘을 같은 작업으로 설명하면 좌표와 검출 면적이 어긋납니다.

스스로 설명할 질문: ROI 자르기와 resize는 어떤 정보를 바꾸나요?

활동 연결: [입력](#) · [기록 양식](#)

A05 · 따라 하기와 작성칸

A05 · 관심 영역과 크기

목적 배경까지 처리하면 관계없는 밝은 물체도 함께 잡힐 수 있습니다. 잘라내기와 크기 변경은 목적과 좌표를 바꾸므로 원본 크기를 기록해야 합니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A05_input.md](#)

작업: [A05_worksheet.md](#)

단계

1. ROI의 시작-끝 번호를 표시합니다.
2. 결과 높이·너비를 계산합니다.
3. 전체 크기를 반으로 바꾼 후 면적 기준을 그대로 써도 될지 토의합니다.

예상 결과 ROI 계산에 예상과 관찰 근거를 남깁니다.

확인 방법 ROI 자르기와 resize는 어떤 정보를 바꾸나요?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A06 · 회색조의 얻는 것과 잃는 것

회색조는 색 성분을 밝기 한 값으로 표현한 이미지입니다. 색을 구분하는 문제인지 밝은 영역을 찾는 문제인지에 따라 사용 할지 정합니다.

밝기 기준으로 물체 영역을 찾는 이번 예에서는 한 채널 비교가 쉽습니다. 그러나 밝기가 비슷한 빨강과 초록을 구분하려면 색 정보를 보존한 방법이 필요합니다.

입력에서 살펴볼 예

BGR: 빨강[0,0,255], 초록[0,255,0], 파랑[255,0,0] 회색조: 76,150,29 별도 색 견본 파일을 비교합니다.

세 채널을 한 밝기로 요약합니다. 빨강·초록·파랑 견본은 76·150·29로 변환됩니다. 검출용 parts.png는 처음부터 세 채널 값이 같으므로 변환 전후 외관이 같습니다.

구분해야 할 반례

회색조가 모든 영상 문제에서 더 좋은 입력은 아닙니다. 신호등 색을 구별해야 하는 업무에서 밝기만 보면 다른 색을 혼동할 수 있습니다.

스스로 설명할 질문: 회색조를 쓰면 어떤 질문에는 답하기 어려워지나요?

활동 연결: [입력](#) · [기록 양식](#)

A06 · 따라 하기와 작성칸

A06 · 회색조의 얻는 것과 잃는 것

목적 밝기 기준으로 물체 영역을 찾는 이번 예에서는 한 채널 비교가 쉽습니다. 그러나 밝기가 비슷한 빨강과 초록을 구분하려면 색 정보를 보존한 방법이 필요합니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A06_input.md](#) · [color-patches.png](#) · [gray-patches.png](#)

작업: [A06_worksheet.md](#)

단계

1. color-patches.png와 gray-patches.png를 비교합니다.
2. 세 BGR 값과 대응 밝기 76·150·29를 연결합니다.
3. 색이 꼭 필요한 반례 하나를 설명합니다.

예상 결과 원본과 변환 비교에 예상과 관찰 근거를 남깁니다.

확인 방법 회색조를 쓰면 어떤 질문에는 답하기 어려워지나요?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

3교시 · 밝기와 연결 영역

13:00~13:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 임계값과 면적 조건

먼저 할 활동: A07, A08, A09

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A07 · 임계값과 이진 마스크

3교시 필수 개념 보충 · 임계값과 면적 기준은 서로 다른 질문이다

실무 상황과 원리

검사 화면에서 밝은 부품 세 개와 작은 먼지 하나가 보입니다. 먼저 밝은 위치를 고르고, 그다음 너무 작은 영역을 제외하려 합니다. 첫 단계는 어떤 픽셀을 남길지 묻고, 두 번째는 남은 덩어리 중 어떤 크기를 셀지 묻습니다. 이 두 질문에 각각 threshold와 min_area라는 설정 이름을 붙입니다.

THRESH_BINARY는 밝기가 임계값보다 클 때 255, 작거나 같을 때 0을 출력합니다. 입력 밝기 [80,100,101,220]과 임계값 100이면 출력은 [0,0,255,255]입니다. 100이 흰색이 되지 않는 이유는 비교가 크거나 같음이 아니라 엄격한 초과이기 때문입니다. 흰색은 조건 통과 위치를 뜻하며 부품 종류나 불량량의 의미까지 알려 주지 않습니다.

윤곽선은 남은 영역의 경계입니다. contourArea는 경계 좌표로 계산한 기하학적 면적이며 흰 픽셀 개수를 그대로 센 값과 같다고 보장하지 않습니다. 면적 [25,100,180]에서 최소 면적 100과 비교 조건 $>=$ 를 쓰면 두 영역이 남습니다. 첫 밝기 기준의 $>$ 와 면적 기준의 $>=$ 가 다르다는 점을 코드에서 찾습니다. 붙어 있는 두 부품은 한 윤곽선으로 연결될 수 있어 영역 수와 실제 부품 수가 달라집니다. 완성 예는 회색조 입력 → 이진 마스크 → 외곽 윤곽선 → 면적 필터 → 남은 영역 수 2입니다. 마스크와 박스 그림을 함께 봐야 숫자가 생긴 이유를 설명할 수 있습니다.

새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 밝기 [60,90,91,200], threshold=90의 마스크를 작성하세요. 이어서 윤곽선 면적 [49,50,120], min_area=50에서 남는 영역 수를 별도로 판단하세요.

정답과 오답의 이유

마스크는 [0,0,255,255], 영역 수는 2입니다. 밝기 90을 남기는 답은 경계 비교 오류이고, 면적 50을 버리는 답은 코드의 >=를 >로 바꿔 읽은 오류입니다. 두 흰 픽셀과 두 영역이 같은 수라는 일반 규칙은 없습니다.

확인 질문과 답

1. threshold의 단위는? → 이 예의 8비트 밝기 값입니다.
2. min_area의 역할은? → 작은 윤곽선 영역을 제외합니다.
3. 붙은 부품 둘은 반드시 두 영역인가요? → 아닙니다.

짧은 완성 코드로 추적하기

제공 참고 파일 [code/05_threshold_trace.py](#)은 보존합니다. 예제 `course` 폴더에서 이 파일을 `work/05_threshold_trace.py`에 복사하고 학생 수정본은 `work`에 저장합니다. 아래는 완결된 전체 코드입니다. 파일·폴더·확장자를 확인한 뒤 같은 `course` 폴더의 터미널에서 `python -X utf8 work/05_threshold_trace.py`을 실행합니다. 저장과 실행은 별도 행동입니다.

```
import cv2

import numpy as np

gray = np.array([[80, 100, 101, 220]], dtype=np.uint8)

used, mask = cv2.threshold(gray, 100, 255, cv2.THRESH_BINARY)

print("threshold:", used)

print("mask:", mask.tolist())
```

예상 출력:

```
threshold: 100.0

mask: [[0, 0, 255, 255]]
```

1행은 영상 도구 `cv2`를 가져옵니다. 2행은 배열 도구 `numpy`를 `np`라는 짧은 이름으로 가져옵니다. 3행의 대괄호 두 겹은 한 행과 네 칸을 가진 배열이며 `dtype=np.uint8`은 0255의 정수로 저장한다는 뜻입니다. 4행은 함수에 입력·기준·최댓값·방식을 전달하고 심표로 반환값 두 개를 나눠 받습니다. 56행의 괄호는 출력 호출, 점은 도구의 기능을 선택하는 기호입니다. `tolist`는 배열을 목록으로 보여 줍니다. `threshold`를 101로 바꾸면 마스크는 `[[0,0,0,255]]`입니다. `cv2` 설치 오류는 환경 문제로 기록하고 제공 출력을 비교합니다. 파일 확장자가 `.py.txt`인지도 확인합니다.

임계값은 값을 나누는 기준이고 마스크는 처리할 영역을 표시한 배열입니다. 출입 기준표처럼 픽셀마다 기준 통과 여부를 정합니다.

`THRESH_BINARY`에서 밝기가 기준보다 크면 255, 작거나 같으면 0입니다. 결과의 흰색은 조건을 만족한 위치이며 물체의 의미를 이미 알아냈다는 뜻은 아닙니다.

입력에서 살펴볼 예

밝기 80,100,101,220 threshold=100 THRESH_BINARY, 최대값255

80과 100은 0입니다. 101과 220은 255입니다. 같은 100을 포함하는 쪽을 주의합니다.

구분해야 할 반례

threshold=100이라고 밝기 100도 흰색이 되지 않습니다. 이 API의 비교는 기준보다 큰 값에 최대값을 줍니다.

스스로 설명할 질문: 밝기 100과 101이 서로 다른 결과가 되는 이유는 무엇인가요?

활동 연결: [입력](#) · [기록 양식](#)

A07 · 따라 하기와 작성칸

A07 · 임계값과 이진 마스크

목적 THRESH_BINARY에서 밝기가 기준보다 크면 255, 작거나 같으면 0입니다. 결과의 흰색은 조건을 만족한 위치이며 물체의 의미를 이미 알아냈다는 뜻은 아닙니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A07_input.md](#)

작업: [A07_worksheet.md](#)

단계

1. 네 밝기 값을 기준과 비교합니다.
2. 예상 마스크 네 값을 씁니다.
3. 기준을 60으로 바꾸면 어떤 영역이 추가되는지 예상합니다.

예상 결과 임계값 손계산에 예상과 관찰 근거를 남깁니다.

확인 방법 밝기 100과 101이 서로 다른 결과가 되는 이유는 무엇인가요?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과-근거: _____

다음에 확인할 것: _____

A08 · 윤곽선과 면적 조건

윤곽선은 연결된 영역의 경계를 따라가는 점들의 목록입니다. 경계 안의 면적은 픽셀 좌표계에서 계산하며 실제 물체의 이름을 알려 주지는 않습니다.

밝은 먼지처럼 작은 영역을 제외하려면 최소 면적 조건을 함께 씁니다. 경계가 합쳐진 두 물체는 하나의 영역으로 나올 수 있으므로 개수의 해석을 제한해야 합니다.

입력에서 살펴볼 예

마스크: 큰 밝은 영역 2개 작은 밝은 영역 1개 min_area=100

작은 영역의 윤곽선 면적은 16으로 제외합니다. 남은 영역을 세면 기본 예에서는 2입니다.

구분해야 할 반례

윤곽선 한 개가 언제나 실제 물체 한 개는 아닙니다. 물체가 붙으면 경계가 합쳐지고 그림자나 반사도 영역을 만들 수 있습니다.

스스로 설명할 질문: 검출 영역 수를 부품 개수라고 바로 확정하면 안 되는 경우는 무엇인가요?

활동 연결: [입력](#) · [기록 양식](#)

A08 · 따라 하기와 작성칸

A08 · 윤곽선과 면적 조건

목적 밝은 먼지처럼 작은 영역을 제외하려면 최소 면적 조건을 함께 씁니다. 경계가 합쳐진 두 물체는 하나의 영역으로 나올 수 있으므로 개수의 해석을 제한해야 합니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A08_input.md](#)

작업: [A08_worksheet.md](#)

단계

1. 큰 영역과 작은 잡음을 구분합니다.
2. 면적 기준 100의 역할을 적습니다.
3. 붙은 부품이나 반사의 반례를 설명합니다.

예상 결과 면적 조건에 예상과 관찰 근거를 남깁니다.

확인 방법 검출 영역 수를 부품 개수라고 바로 확정하면 안 되는 경우는 무엇인가요?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과-근거: _____

다음에 확인할 것: _____

A09 · 설정 파일로 전체 처리 읽기

처리 파이프라인은 입력부터 결과까지 순서대로 이어진 작업입니다. 파일 읽기·회색조·마스크·윤곽선·표시·기록의 연결을 말합니다.

단계를 나누면 결과가 틀렸을 때 어떤 중간 결과를 볼지 정할 수 있습니다. 설정을 코드 밖 JSON에 두면 밝기 기준과 면적 기준을 구분해서 바꿀 수 있습니다.

입력에서 살펴볼 예

02_count.py settings: image=parts.png threshold=100, min_area=100

기본 출력 count는 2입니다. work에 mask.png, detected.png, report.json을 저장합니다.

구분해야 할 반례

프로그램이 정상 종료했다는 사실만으로 검출이 맞는 것은 아닙니다. 입력·마스크·표시된 박스와 원문 조건을 함께 봐야 합니다.

스스로 설명할 질문: 개수가 이상하면 어떤 중간 결과부터 보겠습니까?

활동 연결: [입력](#) · [기록 양식](#)

02_count.py 전체

```
import cv2
import json
from pathlib import Path

root = Path(__file__).resolve().parent.parent
config = json.loads((root / "work" / "settings.json").read_text(encoding="utf-8"))
source = root / "data" / config["image"]
image = cv2.imread(str(source))

if image is None:
    raise ValueError("입력 이미지를 확인하세요: " + str(source))

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
used, mask = cv2.threshold(gray, config["threshold"], 255, cv2.THRESH_BINARY)
contours, hierarchy = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
kept = []

for contour in contours:
    if cv2.contourArea(contour) >= config["min_area"]:
        kept.append(contour)

preview = image.copy()

for contour in kept:
    x, y, width, height = cv2.boundingRect(contour)
    cv2.rectangle(preview, (x, y), (x + width - 1, y + height - 1), (0, 0, 255), 2)

if not cv2.imwrite(str(root / "work" / "mask.png"), mask):
    raise OSError("mask 저장 실패")

if not cv2.imwrite(str(root / "work" / "detected.png"), preview):
    raise OSError("detected 저장 실패")

report = {"image": config["image"], "threshold": used, "count": len(kept)}
(root / "work" / "report.json").write_text(json.dumps(report, ensure_ascii=False), encoding="utf-8")
print(json.dumps(report, ensure_ascii=False))
```

실행 조건: Python 3와 OpenCV가 설치된 PC. 카메라나 GUI 창을 열지 않는 파일 입력 예제입니다.

실행 폴더: examples/course

명령: `python -X utf8 code/02_count.py`

결과:

```
{"image": "parts.png", "threshold": 100.0, "count": 2}
```

각 줄의 뜻은 [코드 해설](#)에 있습니다.

작은 수정: work/settings.json의 threshold만 100에서 60으로 바꾸면 count가 3이 됩니다. min_area는 100을 유지합니다.

오류 복구: 설정의 큰따옴표·솜표 오류는 JSON 오류입니다. 면적 조건과 밝기 조건을 한 번에 바꾸지 말고 원본 설정부터 복구합니다.

A09 · 따라 하기와 작성칸

A09 · 설정 파일로 전체 처리 읽기

목적 단계를 나누면 결과가 틀렸을 때 어떤 중간 결과를 볼지 정할 수 있습니다. 설정을 코드 밖 JSON에 두면 밝기 기준과 면적 기준을 구분해서 바꿀 수 있습니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A09_input.md](#) · [02_count.py](#) · [settings.json](#) · [parts.png](#)

작업: [A09_worksheet.md](#)

단계

1. 설정 파일의 세 항목을 읽습니다.
2. 전체 코드를 단계별로 나누고 기본 결과를 확인합니다.
3. 입력·마스킹·박스·report가 같은 실행의 결과인지 대조합니다.

예상 결과 처리 단계에 예상과 관찰 근거를 남깁니다.

확인 방법 개수가 이상하면 어떤 중간 결과부터 보겠습니까?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

4교시 · 조건 비교와 검출 한계

14:00~14:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 한 조건 비교와 실패

먼저 할 활동: A10, A11, A12

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A10 · 한 조건만 바꾸는 비교

4교시 필수 개념 보충 · 한 조건씩 바꾸고 새 장면으로 시험한다

실무 상황과 원리

밝은 작업대에서는 검출이 잘 되는데 조명을 줄이자 부품 하나가 사라졌습니다. 담당자는 숫자가 맞을 때까지 여러 설정을 동시에 바꾸고 싶습니다. 그러나 원인을 설명하려면 비교 조건을 관리해야 합니다. 같은 사진과 같은 크기를 유지하고 threshold만 바꾸는 실험과 min_area만 바꾸는 실험을 나눕니다.

완성 비교표의 기본 행은 파일 P, threshold 100, min_area 100, 관찰 영역 3입니다. 두 번째 행은 같은 P, threshold 150, min_area 100, 관찰 영역 2라는 가상 관찰입니다. 여기서는 밝기 기준 변화의 영향을 비교할 수 있습니다. 세 번째 행에서 파일까지 어두운 D로 바꾸면 두 변화가 섞여 두 번째 행과 같은 근거로 읽지 못합니다. 값이 달라졌다는 관찰과 왜 달라졌다는 설명을 분리합니다.

기본 파일은 규칙을 만들 때 쓰고, 보류 파일은 아직 설정을 맞추지 않은 새 입력으로 씁니다. 새 입력의 기대 결과를 사람이 먼저 기록한 다음 같은 설정을 적용합니다. 기대 영역 4, 관찰 3이면 그 사례는 실패입니다. 바로 기준을 바꾸었다면 새 설정 버전으로 기존 장면도 다시 시험해야 합니다. 빈 사진, 어두운 사진, 불은 물체 사진은 서로 다른 실패를 드러냅니다. 세 합성 사진 통과를 실제 웹캠 전체 조건의 정확도라고 부르지 않습니다. 각 실행 직후 settings, mask, detected, report를 같은 실험 폴더에 복사하면 뒤 실행이 이전 증거를 덮어쓰는 일을 막을 수 있습니다.

새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 원본 조건 (100,100)에서 한 팀은 (80,50)으로, 다른 팀은 (80,100)으로 바꾸었습니다. 밝기 기준의 영향만 설명할 수 있는 팀과 이유를 쓰고 새 보류 입력 한 종류를 제안하세요.

정답과 오답의 이유

두 번째 팀은 면적 기준을 유지했으므로 밝기 기준 비교에 적합합니다. 첫 팀의 결과에는 두 설정 변화가 섞였습니다. 다른 배치나 다른 조명 사진을 제안하되 무엇을 확인하고 무엇을 아직 확인하지 못하는지 적으면 인정합니다.

확인 질문과 답

1. 기대값은 언제 쓰나요? → 실행 전에 씁니다.
2. 새 설정이면 무엇을 재시험하나요? → 기존 입력과 새 입력을 함께 봅니다.

3. 파일 통과가 실물 통과인가요? → 아닙니다.

비교 실험은 바꾼 조건과 고정한 조건을 구분해 결과 차이의 원인을 살피는 활동입니다. 사진 두 장을 비교할 때 조명까지 함께 바뀌면 이유를 단정하기 어렵습니다.

threshold와 min_area를 동시에 바꾸면 어느 조건 때문에 개수가 달라졌는지 알기 어렵습니다. 기본값을 적고 한 값씩 수정한 뒤 원래 값으로 돌립니다.

입력에서 살펴볼 예

실험1 threshold100, area100 실험2 threshold60, area100 실험3 threshold100, area2000

개수 예상은 2,3,1입니다. 실험3은 두 번째 큰 영역만 남깁니다.

구분해야 할 반례

실험2 결과가 좋아 보인다고 모든 카메라 조건에서 60이 최적이라는 뜻은 아닙니다. 비교에 쓰지 않은 조명과 배경에서도 확인해야 합니다.

스스로 설명할 질문: 어느 값을 고정해야 변경의 영향을 설명할 수 있나요?

활동 연결: [입력](#) · [기록 양식](#)

A10 · 따라 하기와 작성칸

A10 · 한 조건만 바꾸는 비교

목적 threshold와 min_area를 동시에 바꾸면 어느 조건 때문에 개수가 달라졌는지 알기 어렵습니다. 기본값을 적고 한 값씩 수정한 뒤 원래 값으로 돌립니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A10_input.md](#)

작업: [A10_worksheet.md](#)

단계

1. 세 설정의 결과를 먼저 예상합니다.
2. 한 값씩 바꾸고 출력과 그림을 기록합니다.
3. 원래 설정으로 복구하고 재현되는지 확인합니다.

예상 결과 조건 비교표에 예상과 관찰 근거를 남깁니다.

확인 방법 어느 값을 고정해야 변경의 영향을 설명할 수 있나요?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

각 실험의 파일을 바로 보관

실험 1 직후 `work/experiment-1` 폴더를 만들고 사용한 `settings.json`과 이번 `mask.png`, `detected.png`, `report.json`을 복사합니다. `report.json`의 `threshold-min_area`를 사용 설정과 대조한 뒤 조건을 바꿉니다. 실험 2·3도 `experiment-2`, `experiment-3`에 각각 보관합니다. 다음 실행은 같은 출력 파일을 덮어쓰므로 마지막 출력만 세 실험 결과로 제출하지 않습니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A11 · 실패한 검출에서 배우기

오검출은 대상이 아닌 영역을 대상으로 판단한 경우이고 미검출은 실제 목표를 놓친 경우입니다. 먼저 업무에서 무엇을 대상이라고 정했는지 있어야 두 오류를 셀 수 있습니다.

빈 장면, 어두운 장면, 붙은 장면을 따로 시험하면 기본 이미지에서 보이지 않던 한계를 확인할 수 있습니다. 사람의 원문 판정과 알고리즘 출력은 서로 다른 근거입니다.

입력에서 살펴볼 예

`empty.png`: 대상0 `dim.png`: 같은2개가 어두워짐 `touching.png`: 실제2개가 붙음

기준100에서 `empty=0`, `dim=0`, `touching=1`입니다. 붙은 영역 한 개가 실제 부품 두 개일 수 있습니다.

구분해야 할 반례

전체 결과를 정확도 한 숫자로만 쓰면 어떤 장면에서 실패했는지 숨길 수 있습니다. 작은 수업 샘플의 통과율도 현장 성능 증명은 아닙니다.

스스로 설명할 질문: 빈 장면 통과와 어두운 장면 실패를 어떻게 따로 기록하나요?

활동 연결: [입력](#) · [기록 양식](#)

A11 · 따라 하기와 작성칸

A11 · 실패한 검출에서 배우기

목적 빈 장면, 어두운 장면, 붙은 장면을 따로 시험하면 기본 이미지에서 보이지 않던 한계를 확인할 수 있습니다. 사람의 원문 판정과 알고리즘 출력은 서로 다른 근거입니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A11_input.md](#) · [empty.png](#) · [dim.png](#) · [touching.png](#)

작업: [A11_worksheet.md](#)

단계

1. 각 파일의 실제 목표 수를 읽습니다.
2. 같은 설정으로 관찰 결과를 기록합니다.
3. 오류 원인과 추가로 필요한 입력을 설명합니다.

예상 결과 장면별 예상과 실제에 예상과 관찰 근거를 남깁니다.

확인 방법 빈 장면 통과와 어두운 장면 실패를 어떻게 따로 기록하나요?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과-근거: _____

다음에 확인할 것: _____

A12 · 새 입력으로 규칙 확인

보류 입력은 기준을 정할 때 미리 맞춰 보지 않은 입력입니다. 연습 문제의 답에만 맞춘 규칙인지 확인할 기회를 줍니다.

같은 그림의 숫자만 외우면 다른 배치에서 틀려도 놓치기 쉽습니다. 새 입력에서는 먼저 기대 결과와 이유를 쓰고 그다음 같은 설정을 적용합니다.

입력에서 살펴볼 예

holdout.png 밝은 분리 영역3개 threshold100, area100 유지

새 위치에 놓인 세 영역이 남습니다. 입력 배치가 바뀌어도 이번 조건에서는 3입니다.

구분해야 할 반례

새 입력 한 장에서 성공했다고 색과 조명 변화까지 모두 해결한 것은 아닙니다. 검증한 변화가 위치뿐인지 밝기·겹침도 포함하는지 기록합니다.

스스로 설명할 질문: 이번 보류 입력이 확인한 변화와 확인하지 못한 변화는 무엇인가요?

활동 연결: [입력](#) · [기록 양식](#)

A12 · 따라 하기와 작성칸

A12 · 새 입력으로 규칙 확인

목적 같은 그림의 숫자만 외우면 다른 배치에서 틀려도 놓치기 쉽습니다. 새 입력에서는 먼저 기대 결과와 이유를 쓰고 그다음 같은 설정을 적용합니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A12_input.md](#) · [holdout.png](#)

작업: [A12_worksheet.md](#)

단계

1. 새 그림을 눈으로 보고 기대 영역 수를 기록합니다.
2. 기본 설정을 유지하고 처리합니다.
3. 확인한 변화와 남은 한계를 나눠 씁니다.

예상 결과 보류 입력 검증에 예상과 관찰 근거를 남깁니다.

확인 방법 이번 보류 입력이 확인한 변화와 확인하지 못한 변화는 무엇인가요?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

5교시 · 프레임 반복과 웹캠

15:00~15:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 프레임 반복과 장치 종료

먼저 할 활동: A13, A14, A15

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A13 · 프레임을 순서대로 처리

5교시 필수 개념 보충 · 프레임마다 초기화와 장치 자원 종료

실무 상황과 원리

같은 부품이 세 프레임에 계속 보이면 검출 결과가 세 번 나옵니다. 이를 물품 세 개가 지나갔다고 보고하면 수량이 틀립니다. 프레임은 한 시점의 사진이며, 프레임별 영역 수는 그 시점에 보이는 영역 수입니다. 고유 물체의 이동과 통과를 세려면 별도의 추적 규칙과 시험이 필요합니다.

제공 파일 재생 예(code/03_replay.py)의 기본 설정에서 parts.png, empty.png, holdout.png의 장면별 검출 수는 [2,0,3]입니다. 반복마다 count=0을 먼저 만들고 해당 장면의 영역만 더하면 출력은 2,0,3입니다. count를 반복 밖에서 한 번만 만들면 누적은 2,2,5가 됩니다. 누적 합이 필요한 업무라면 누적이 맞을 수 있지만 현재 예의 요구는 장면별 수이므로 초기화 위치가 중요합니다. 이 개수는 해당 입력과 설정에서 얻은 영역 수이며 물품의 실제 수량을 보장하지 않습니다. for는 파일 목록에서 한 이름씩 꺼내고, 들여쓰기는 그 이름을 처리하는 문장 묶음을 표시합니다.

실제 카메라 경로에서는 VideoCapture로 장치를 열고 isOpened로 열기 성공을 확인합니다. read의 ok는 프레임을 받았는지 알리는 값입니다. 장치 열기 성공과 계속 프레임을 받는 성공은 별도입니다. 실패한 프레임에 바로 색 변환을 적용하지 않습니다. finally는 정상 종료와 오류 종료 뒤 모두 자원을 정리할 자리를 제공합니다. release는 카메라 점유를 풀고 destroyAllWindows는 만든 창을 닫습니다. 파일 재생이 완료되어도 권한, 장치 번호, GUI 창, 프레임 지연은 확인하지 않은 상태입니다. 실제 장비가 없으면 장치를 임의로 열지 않고 종료 흐름을 종이에서 설명합니다.

새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: 프레임의 영역 수가 [2,1,0]일 때 매년 초기화한 출력과 누적 출력의 차이를 직접 쓰세요. 카메라 열기 성공 뒤 read가 실패하면 다음 행동도 판단하세요.

정답과 오답의 이유

프레임별 출력은 2,1,0이고 누적은 2,3,3입니다. read 실패이면 처리 반복을 끝내고 자원을 해제합니다. 마지막 정상 프레임은 새 측정처럼 반복 출력하면 관찰 시각이 틀린 보고가 됩니다.

확인 질문과 답

1. 프레임 세 장은 물품 세 개인가요? → 그렇게 단정할 수 없습니다.
2. count=0 위치는? → 프레임 처리 반복 안입니다.

3. finally의 목적은? → 종료 경로에서 장치와 창을 정리합니다.

반복은 같은 처리 규칙을 여러 입력에 차례로 적용하는 방법입니다. 프레임마다 개수를 다시 세는 일과 시간 전체의 고유 물체를 추적하는 일은 다릅니다.

파일 목록을 재생하면 카메라 없이도 반복·초기화·출력 순서를 확인할 수 있습니다. 루프 안에서 count를 0으로 다시 만드는 이유를 설명할 수 있어야 합니다.

입력에서 살펴볼 예

03_replay.py parts, empty, holdout 순서 기대 영역 수 2,0,3

파일 이름과 해당 프레임의 개수가 줄마다 나옵니다. 2+0+3을 고유 부품 총수로 해석하지 않습니다.

구분해야 할 반례

카메라가 초당 30프레임을 준다고 프로그램도 반드시 초당30번 처리를 끝내는 것은 아닙니다. 처리시간과 장치의 출력속도는 별도로 측정해야 합니다.

스스로 설명할 질문: count=0을 반복문 밖으로 옮기면 어떤 문제가 생기나요?

활동 연결: [입력](#) · [기록 양식](#)

03_replay.py 전체

```
import cv2

from pathlib import Path

root = Path(__file__).resolve().parent.parent

for name in ["parts.png", "empty.png", "holdout.png"]:
    frame = cv2.imread(str(root / "data" / name))

    if frame is None:
        raise ValueError("프레임을 읽을 수 없습니다: " + name)

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    used, mask = cv2.threshold(gray, 100, 255, cv2.THRESH_BINARY)

    contours, hierarchy = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    count = 0

    for contour in contours:
        if cv2.contourArea(contour) >= 100:
            count = count + 1

    print(name, count)
```

실행 조건: Python 3와 OpenCV가 설치된 PC. 카메라나 GUI 창을 열지 않는 파일 입력 예제입니다.

실행 폴더: examples/course

명령: `python -X utf8 code/03_replay.py`

결과:

```
parts.png 2
empty.png 0
holdout.png 3
```

각 줄의 뜻은 [코드 해설](#)에 있습니다.

작은 수정: 목록 순서만 바꾸면 출력 순서만 달라집니다. 이것은 프레임별 개수이며 고유 물체 누적 수가 아닙니다.

오류 복구: 이전 프레임 개수가 다음 프레임에 더해지면 count = 0이 반복문 안에 있는지 확인합니다.

A13 · 따라 하기와 작성칸

A13 · 프레임을 순서대로 처리

목적 파일 목록을 재생하면 카메라 없이도 반복·초기화·출력 순서를 확인할 수 있습니다. 루프 안에서 count를 0으로 다시 만드는 이유를 설명할 수 있어야 합니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A13_input.md](#) · [03_replay.py](#)

작업: [A13_worksheet.md](#)

단계

1. 전체 코드의 반복 범위를 표시합니다.
2. 출력 세 줄을 예상하고 실제 결과와 비교합니다.
3. 목록 순서를 바꾸고 값과 순서의 변화를 적습니다.

예상 결과 프레임별 출력에 예상과 관찰 근거를 남깁니다.

확인 방법 count=0을 반복문 밖으로 옮기면 어떤 문제가 생기나요?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A14 · 웹캠 열기와 종료

카메라 캡처는 장치에서 프레임을 받아오는 연결입니다. 장비를 빌려 쓰듯 열기와 사용 뒤 반환이 한 묶음이어야 합니다.

다른 앱이 카메라를 쓰거나 권한이 없으면 열기가 실패할 수 있습니다. 열기 성공과 매 프레임 읽기 성공을 따로 검사하고 모든 종료 경로에서 자원을 해제합니다.

입력에서 살펴볼 예

04_webcam.py 열기 확인 isOpened 프레임 확인 read 종료 q와 finally

기본 제작은 카메라를 열지 않았습니다. 코드 읽기와 모의 실패 경로를 비교합니다.

구분해야 할 반례

장치 번호0이 항상 원하는 카메라라는 보장은 없습니다. 번호를 무작정 바꿔 촬영하기 전에 장치·권한·촬영 범위를 강사가 확인합니다.

스스로 설명할 질문: 열기는 성공했는데 read가 실패할 수 있을까요?

활동 연결: [입력](#) · [기록 양식](#)

04_webcam.py 전체

```
import cv2

capture = cv2.VideoCapture(0)

try:
    if not capture.isOpened():
        raise RuntimeError("카메라 열기 실패: 권한, 점유, 장치 번호를 확인하세요")
    while True:
        ok, frame = capture.read()
        if not ok:
            print("프레임 읽기 실패: 종료합니다")
            break
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        cv2.imshow("camera-gray", gray)
        if cv2.waitKey(1) & 255 == ord("q"):
            break
finally:
    capture.release()
    cv2.destroyAllWindows()
```

실행 조건: 선택 실습: 강사가 확인한 웹캠·권한·GUI 지원 OpenCV와 사용자 동의가 있는 촬영 범위. 이번 제작에서는 실제 카메라를 열지 않았습니다.

실행 폴더: examples/course

명령: `python -X utf8 code/04_webcam.py`

결과:

예상: 회색조 창이 갱신되고 창이 활성화된 상태에서 q를 누르면 종료.

실제 장치·창 실행은 미수행. 모의 캡처로 열기 실패·읽기 실패·q 종료와 release 호출을 검증.

각 줄의 뜻은 [코드 해설](#)에 있습니다.

작은 수정: 0을 임의 번호로 계속 바꾸기 전에 운영체제의 카메라 목록과 권한을 확인합니다. 기본 실습은 03_replay.py로 대신합니다.

오류 복구: 장치 점유 프로그램을 종료하고 권한·번호를 확인합니다. GUI가 없는 환경에서는 imshow 대신 파일 경로로 진행합니다.

A14 · 따라 하기와 작성칸

A14 · 웹캠 열기와 종료

목적 다른 앱이 카메라를 쓰거나 권한이 없으면 열기가 실패할 수 있습니다. 열기 성공과 매 프레임 읽기 성공을 따로 검사하고 모든 종료 경로에서 자원을 해제합니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A14_input.md](#) · [04_webcam.py](#)

작업: [A14_worksheet.md](#)

단계

1. 코드에서 열기·읽기·처리·종료를 표시합니다.
2. 열기 실패와 읽기 실패의 다른 경로를 따릅니다.
3. finally에서 release가 필요한 이유를 설명합니다.

예상 결과 장치 생명주기에 예상과 관찰 근거를 남깁니다.

확인 방법 열기는 성공했는데 read가 실패할 수 있을까요?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A15 · 시연과 실제 장비 검증

검증 경계는 무엇을 직접 실행했고 무엇을 조건으로 남겼는지 나누는 선입니다. 파일과 모의 캡처가 통과한 사실은 실제 하드웨어 동작과 구별해서 기록합니다.

장비가 없는 날에도 실패 흐름을 읽고 시험할 수 있습니다. 다만 드라이버·카메라 권한·실제 GUI·지연은 현장에서 확인할 항목으로 남겨야 합니다.

입력에서 살펴볼 예

모의1: 열기 실패 모의2: 프레임 실패 모의3: 정상 프레임 후 q

세 경로 모두 release 호출을 확인합니다. 실제 카메라 영상 품질과 속도는 미확인입니다.

구분해야 할 반례

함수가 호출되었다는 기록만으로 실제 장치가 물리적으로 해제됐다고 주장하지 않습니다. 모의 객체는 코드의 경로만 대신 합니다.

스스로 설명할 질문: 모의 시험에서 확인한 사실을 한 문장으로 제한해 말해 보세요.

활동 연결: [입력](#) · [기록 양식](#)

A15 · 따라 하기와 작성칸

A15 · 시연과 실제 장비 검증

목적 장비가 없는 날에도 실패 흐름을 읽고 시험할 수 있습니다. 다만 드라이버·카메라 권한·실제 GUI·지연은 현장에서 확인할 항목으로 남겨야 합니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A15_input.md](#) · [webcam-test-cases.md](#)

작업: [A15_worksheet.md](#)

단계

1. 모의 결과 카드의 세 경로를 비교합니다.
2. 각 경로의 종료와 정리 호출을 표시합니다.
3. 현장 점검 체크 항목을 작성합니다.

예상 결과 검증 경계에 예상과 관찰 근거를 남깁니다.

확인 방법 모의 시험에서 확인한 사실을 한 문장으로 제한해 말해 보세요.

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

6교시 · 새 요구와 결과 검수

16:00~16:50 · 50분

설명 10분 · 시연 8분 · 함께 수행 10분 · 개별 수행 15분 · 풀이·확인 7분

이번 교시에서 확인할 것. 요구와 결과 묶음

먼저 할 활동: A16, A17, A18

추가·복습: 새 입력을 추가해 같은 방법으로 확인 · 필수 결과를 확인한 뒤 진행합니다.

A16 · 검출 요구를 문장으로 정하기

6교시 필수 개념 보충 · 검출 요구와 결과 보고를 연결하기

실무 상황과 원리

현장 담당자가 “잘 세어 주세요”라고 요청하면 개발자는 어떤 장면에서 몇 개를 어떻게 셀지 알 수 없습니다. 요구사항은 확인 가능한 입력 조건과 결과의 약속입니다. “분리된 밝은 부품의 외곽 영역을 세고, 입력 파일명과 설정을 함께 저장한다”처럼 관찰 대상과 출력부터 정합니다.

완성 예의 요구 R1은 합성 파일 H에서 사람이 지정한 영역 4개와 프로그램 수를 비교하는 것입니다. 시험 T1의 입력은 H와 설정 (100,100), 기대값은 4입니다. 관찰이 4면 그 시험은 통과이고, 3이면 실패입니다. 실물 카메라 시험 T2는 장치가 없어 미실행이라고 씁니다. R1의 파일 통과가 T2의 미실행을 덮지 않습니다. 숫자 일치 외에 박스가 어떤 영역을 둘렀는지도 살펴보면 반사와 누락이 우연히 서로 상쇄된 경우를 발견할 수 있습니다.

검출 수를 바르게 해석하려면 가정도 붙입니다. 촬영 크기 고정, 밝은 대상, 대상 사이 간격, 관심 영역, 조명 조건은 제작 가정입니다. 대상이 불거나 어두워지면 이 단순 규칙이 실패할 수 있습니다. 결과 묶음에는 원본 이미지, 설정, 처리 명령, 마스크, 박스 그림, 수치 보고와 기대값 비교표를 넣습니다. 동료는 같은 파일과 명령으로 수를 다시 얻고 조건을 읽을 수 있어야 합니다. 파일 작성 완료, 로컬 실행 완료, 실물 수용 확인은 서로 다른 상태입니다. 새로운 요구가 들어오면 조건과 기대값을 먼저 바꾸고 시험 결과를 새 버전으로 남깁니다.

새로운 입력에 직접 적용 · 복습 또는 도전

복습 문제: “컨베이어의 모든 부품을 100% 검출”이라는 요청을 현재 합성 파일 범위에 맞는 작은 요구 하나와 미확인 항목 두 개로 다시 작성하세요.

정답과 오답의 이유

예: “고정 크기의 제공 사진 H에서 분리된 밝은 영역 4개를 세고 설정을 기록한다.” 실제 속도와 조명 변화는 미확인입니다. 4개 일치만으로 모든 현장 장면의 정확도를 100%라 쓰면 시험 범위를 확대해 해석한 오답입니다.

확인 질문과 답

1. 통과와 단위는? → 특정 입력과 기대값을 가진 시험입니다.
2. 박스 그림도 보는 이유는? → 어떤 영역을 썼는지 확인하려고입니다.
3. 장치가 없었던 시험 상태는? → 미실행입니다.

요구사항은 어떤 입력에서 어떤 결과를 기대하는지 정한 약속입니다. 잘 찾는 프로그램처럼 모호한 표현 대신 조건과 확인 기준을 함께 씁니다.

이번 작은 작업은 밝은 분리 영역을 세는 규칙입니다. 색상 분류·사람 인식·실제 부품 품질검사까지 한 번에 수행한다고 쓰면 검증 범위를 벗어납니다.

입력에서 살펴볼 예

새 의뢰: 검은 받침 위 밝은 표식 수 대상: 분리된 영역, 면적100이상 제외: 실제 mm측정·붙은 물체 분리

입력 조건과 결과 항목을 먼저 씁니다. 빈장면·새배치·어두운장면을 시험에 포함합니다.

구분해야 할 반례

요구에 없는 실제 부품 종류 이름을 알고리즘 결과에 붙이지 않습니다. 영역의 밝기·면적 조건만으로 재질이나 불량 원인을 알 수 없습니다.

스스로 설명할 질문: 이번 코드로 약속할 수 있는 결과와 약속하기 어려운 것은 무엇인가요?

활동 연결: [입력](#) · [기록 양식](#)

A16 · 따라 하기와 작성칸

A16 · 검출 요구를 문장으로 정하기

목적 이번 작은 작업은 밝은 분리 영역을 세는 규칙입니다. 색상 분류·사람 인식·실제 부품 품질검사까지 한 번에 수행한다고 쓰면 검증 범위를 벗어납니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A16_input.md](#)

작업: [A16_worksheet.md](#)

단계

1. 목표와 제외 범위를 구분합니다.
2. 입력 조건과 예상 결과를 문장으로 씁니다.
3. 성공 사례와 실패 사례를 하나씩 정합니다.

예상 결과 검출 요구 문장에 예상과 관찰 근거를 남깁니다.

확인 방법 이번 코드로 약속할 수 있는 결과와 약속하기 어려운 것은 무엇인가요?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A17 · 새 설정으로 결과 패키지 만들기

결과 패키지는 원본 이름, 설정, 출력, 확인한 사실을 함께 남긴 묶음입니다. 그림 하나만 보내면 어떤 조건으로 만들었는지 재현하기 어렵습니다.

다음 사람이 같은 결과를 확인하려면 입력과 설정이 필요합니다. 오류 입력에서는 그럴듯한 빈 결과를 성공으로 저장하지 않고 실패 이유를 분리해 남깁니다.

입력에서 살펴볼 예

종합 입력: holdout.png threshold=100, min_area=100 추가 시험: missing.png

정상 입력의 기대 count는 3입니다. 없는 파일은 읽기 오류로 종료해야 합니다.

구분해야 할 반례

입력을 못 읽은 상태와 정상적으로 물체가 없는 상태는 다릅니다. 둘 다 count0으로 기록하면 센서 장애를 빈 공간으로 오해할 수 있습니다.

스스로 설명할 질문: 입력 실패와 정상 빈장면을 어떤 항목으로 구별할까요?

활동 연결: [입력](#) · [기록 양식](#)

A17 · 따라 하기와 작성칸

A17 · 새 설정으로 결과 패키지 만들기

목적 다음 사람이 같은 결과를 확인하려면 입력과 설정이 필요합니다. 오류 입력에서는 그럴듯한 빈 결과를 성공으로 저장하지 않고 실패 이유를 분리해 남깁니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A17_input.md](#) · [holdout.png](#) · [empty.png](#)

작업: [A17_worksheet.md](#)

단계

1. 설정과 기대 결과를 먼저 기록합니다.
2. 새 입력을 실행 또는 제공 결과와 비교합니다.
3. 없는 파일 오류와 빈장면0을 따로 정리합니다.

예상 결과 종합 결과표에 예상과 관찰 근거를 남깁니다.

확인 방법 입력 실패와 정상 빈장면을 어떤 항목으로 구별할까요?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

A18 · 결과를 근거로 설명하기

인수인계는 다음 사람이 작업을 이어갈 수 있도록 입력과 확인 결과, 남은 한계를 전달하는 일입니다. 기능 소개와 실제 검증 결과를 구분해 말합니다.

PPT 장면이나 출력 숫자만 보여 주는 발표보다 어느 조건에서 성공·실패했는지를 설명해야 수정 방향을 정할 수 있습니다. 기록의 날짜와 사용 설정도 함께 남깁니다.

입력에서 살펴볼 예

최종 설명3문장 1 입력과 규칙 2 직접 확인한 결과 3 미검증 장비 조건

파일 입력은 실제 실행 근거를 제시합니다. 카메라·GUI·현장 성능은 미실행 경계로 설명합니다.

구분해야 할 반례

완성 코드가 있다는 이유로 실시간 품질검사 시스템이 완성됐다고 말하지 않습니다. 교육용 합성 입력과 현장 운영은 검증 범위가 다릅니다.

스스로 설명할 질문: 확인한 사실과 다음 검증을 각각 한 문장으로 말할 수 있나요?

활동 연결: [입력](#) · [기록 양식](#)

A18 · 따라 하기와 작성칸

A18 · 결과를 근거로 설명하기

목적 PPT 장면이나 출력 숫자만 보여 주는 발표보다 어느 조건에서 성공·실패했는지를 설명해야 수정 방향을 정할 수 있습니다. 기록의 날짜와 사용 설정도 함께 남깁니다.

준비 입력 파일과 해당 work 기록 양식을 열고 원문은 보존합니다. 코드 활동은 설치 조건을 확인하거나 제공 출력 비교로 진행합니다.

입력: [A18_input.md](#)

작업: [A18_worksheet.md](#)

단계

1. 원본·설정·출력 파일을 제출 목록에 적습니다.
2. 세 문장으로 검증 결과와 한계를 설명합니다.
3. 새 질문 네 개에 근거를 답합니다.

예상 결과 제출 목록에 예상과 관찰 근거를 남깁니다.

확인 방법 확인한 사실과 다음 검증을 각각 한 문장으로 말할 수 있나요?

오류 해결 입력 파일과 변경한 값 하나를 확인하고 원본 설정으로 돌아가 비교합니다.

힌트: 전체를 다시 쓰기 전에 원문의 해당 값이나 코드 줄을 함께 찾습니다.

선택 확장: 조건 하나가 달라진 새 입력을 만들어 예상 결과와 이유를 설명합니다.

수행 기록: ☐ 개인 실행 ☐ 강사 실행 관찰 ☐ 제공 출력 비교 ☐ 수동 추적

수정 전 예상: _____

실제 결과·근거: _____

다음에 확인할 것: _____

용어와 코드 기호 찾아보기

용어를 처음 만날 때

OpenCV

이미지·영상 읽기와 처리를 제공하는 라이브러리. 사진 의미를 자동으로 모두 이해하는 AI는 아닙니다.

픽셀

한 위치의 밝기 또는 색 값을 담는 이미지 요소. 현실의 고정 mm 길이와 다릅니다.

좌표

위치 번호. 화면의 x,y와 배열의 [y,x] 순서를 구분합니다.

BGR

파랑·초록·빨강 채널 순서. OpenCV의 기본 컬러 읽기 약속입니다.

shape

배열의 각 차원 크기. 컬러 이미지는 높이·너비·채널 수로 읽습니다.

ROI

관심 영역. 분석할 이미지의 일부를 고릅니다.

회색조

색 세 성분을 밝기 한 값으로 표현한 배열. 색 정보가 줄어듭니다.

임계값

크다/작다를 가르는 기준. 이번 이진화는 기준보다 큰 밝기를 255로 만듭니다.

마스크

남길 위치를 표시하는 배열. 여기서는 0과255로 표시합니다.

윤곽선

연결된 영역의 경계를 따라간 점의 목록. 실제 물체와 항상 일대일은 아닙니다.

프레임

영상의 한 시점 이미지. 같은 물체가 여러 프레임에 반복됩니다.

자원 해제

사용을 마친 카메라 같은 자원을 반환하는 일. 코드의 모든 종료 경로에 필요합니다.

모의 객체

실제 장치 대신 성공·실패를 정해 반환하는 시험용 대역. 현장 품질 검증과 다릅니다.

오늘 배운 내용 확인하기

평가 · OpenCV와 웹캠

이 평가는 이미지를 처리한 결과를 근거로 설명할 수 있는지 확인합니다. 코드를 외우거나 빨리 입력하는 능력은 평가하지 않습니다. 직접 실행하기 어려우면 제공 출력과 강사 시연을 비교하고, 기록에 수행 방법을 표시하세요.

준비와 제출 방법

1. 실습에서 작성한 `work/A17_worksheet.md`와 `work/A18_worksheet.md`를 엽니다. 여기서 `work`는 예제 ZIP을 푼 폴더 안의 작업 폴더입니다.
2. A17 기록에 사용한 입력 파일 이름, 임계값, 최소 면적, 실행 전 예상, 실제 검출 수를 적습니다. 임계값은 밝고 어두운 픽셀을 나누는 기준이고, 최소 면적은 너무 작은 영역을 제외하는 기준입니다.
3. 직접 생성한 출력이 있다면 설정 파일과 회색조·이진화·검출 결과 이미지를 함께 제출합니다. 제공 출력만 비교했다면 새로 실행했다고 쓰지 말고, 비교한 파일 이름과 관찰을 적습니다.
4. A18에는 아래 세 문장을 자신의 결과에 맞게 완성합니다. "이 입력에서 **을 찾으려고 했다.**" "설정을 사용하자 ____ 결과가 나왔다." "이 결과만으로 ____까지 확인한 것은 아니다."

제출 묶음은 **A17 결과표 + 설정 값 + 출력 또는 제공 출력 대조 기록 + A18 설명**입니다. 실제 카메라가 없어도 파일 실습과 모의 장치 흐름의 근거로 평가받을 수 있습니다.

수행 과제

- 이미지의 높이·너비·채널을 읽고, 한 픽셀을 `[y, x]`로 찾는 이유를 설명하세요. BGR은 파랑·초록·빨강 순서의 색 값이라는 점을 예시와 연결하세요.

- 임계값 100과 60을 비교한 기록, 최소 면적을 바꾼 기록을 제시하세요. 한 번에 바꾼 설정과 그대로 둔 설정을 구별하세요.
- 빈 장면, 어두운 장면, 표식이 붙은 장면, 처음 보는 입력에서 예상과 실재를 비교하세요. "읽기에 실패함"과 "정상적으로 읽었지만 검출 수가 0임"을 같은 결과로 쓰지 마세요.
- 카메라 예제의 열기 → 읽기 → 종료 입력 → 자원 정리 흐름을 말로 설명하세요. 모의 시험에서 확인한 것과 실제 카메라에서 아직 확인하지 않은 것을 따로 적으세요.

채점 기준 · 100점

관점	배점	충분한 근거의 기준
이미지 기초	25	높이·너비·BGR·[y, x]를 실제 예시의 값과 연결하여 설명한다.
처리와 비교	30	입력 이름과 설정을 남기고, 임계값·면적 변경 전후의 예상과 실제 결과를 비교한다.
검수와 한계	25	빈·어두운·붙은·새 입력의 차이를 설명하고, 읽기 실패와 정상 0을 구분한다.
장치 흐름 설명	20	열기·읽기·q·finally·release의 역할과 실제 미검증 범위를 설명한다.

각 관점은 **근거 없음 0% / 일부 근거가 있으나 핵심 누락 50% / 입력·설정·출력이 맞고 한계까지 설명 100%**로 채점합니다. 예를 들어 30점 항목은 0.15·30점입니다. 25점 항목의 중간 점수는 12.5점입니다. 70점 이상이고 입력 실패와 정상 빈 장면을 구분하면 기본 목표에 도달한 것으로 봅니다.

설명으로 확인하는 질문

1. `shape`가 `(480, 640, 3)`이라면 너비와 채널 수는 각각 무엇인가요? 숫자의 순서를 함께 설명하세요.
2. `THRESH_BINARY`의 기준이 100일 때 밝기가 정확히 100인 픽셀은 어느 쪽으로 처리되나요? 코드의 비교 규칙을 근거로 설명하세요.
3. 붙어 있는 표식 두 개가 윤곽선 하나로 검출됐다면, 검출 수만 보고 물체가 하나라고 결론 내릴 수 있나요? 다음에 확인할 것을 적으세요.
4. 모의 캡처에서 `release` 호출을 확인하면 실제 카메라 드라이버 동작까지 검증한 것인가요? 시험 범위를 설명하세요.

제출 전 확인

- 파일 이름과 설정이 기록에 남아 있는가?
- 예상과 실재를 구분했는가?
- 직접 실행과 제공 출력 관찰을 구분했는가?
- 실패 사례와 아직 확인하지 않은 조건을 적었는가?

점수가 낮은 항목은 전체를 다시 만들기보다 빠진 입력·설정·관찰 근거를 보충한 뒤 다시 설명합니다. 정답과 채점 예시는 강사용 자료에 있습니다.

새 상황으로 옮겨 보는 전이 문제 · 복습·도전

교시당 한 문제이며 기존 필수 평가 후 필요에 따라 선택합니다. 독립 작성 또는 판단을 먼저 기록한 뒤 해설을 확인합니다. 설치 실패나 장비 부재는 개념 오답과 구분합니다.

전이 1 · 1교시

가로 7·세로 2·채널 3인 사진에서 $x=6, y=1$ 의 표기와 색 성분 수를 쓰세요.

정답과 해설

`image[1,6]`, 42개입니다. 가로·세로 순서를 뒤집거나 42를 물체 수로 쓰지 않습니다.

전이 2 · 2교시

ROI가 `[5:25,10:60]`일 때 크기와 ROI (3,4)의 원본 (x,y)를 계산하세요.

정답과 해설

높이 20·너비 50, 원본 (13,9)입니다. 끝 번호는 제외하고 원본 시작점만 더합니다.

전이 3 · 3교시

밝기 `[99,100,101]`, `threshold=100`과 면적 `[99,100,101]`, `min_area=100`을 각각 판정하세요.

정답과 해설

마스크 `[0,0,255]`, 남는 영역 2개입니다. 밝기의 `>`와 면적의 `>=`를 구별합니다.

전이 4 · 4교시

밝기와 면적 기준을 동시에 바꿔 수가 4에서 3이 되었습니다. 원인 검증용 다음 시험을 작성하세요.

정답과 해설

같은 파일과 면적 기준을 고정하고 밝기만 바꾸거나 반대로 한 조건만 비교합니다. 현재 숫자만으로 원인을 확정할 수 없습니다.

전이 5 · 5교시

한 물체가 10프레임에서 계속 검출됐습니다. 통과 물품 수를 10이라 보고해도 될까요?

정답과 해설

아닙니다. 프레임별 관찰과 고유 물체를 구별해야 하며 이동 추적·통과 기준·중복 시험이 필요합니다.

전이 6 · 6교시

합성 사진 두 개에서 예상 수가 맞았습니다. 실물 컨베이어 인수 보고의 상태를 쓰세요.

정답과 해설

해당 파일 비교는 통과, 실물 속도·조명·장치 시험은 미실행입니다. 합성 입력 결과를 전체 현장 정확도로 확대하지 않습니다.

파일과 다음 수업

링크를 눌러 파일을 내려받습니다. 예제 ZIP은 압축을 풀고 README부터 읽습니다. 원본 code·data는 보존하고 work의 작업본을 수정합니다. 파일이 저장된 폴더와 확장명을 확인하세요.

실습 예제 코드와 데이터

OpenCV + 웹캠 실습 PPT

처음 시작 안내

용어와 코드 기호

학생용 교안

학생 실습 안내

확인 문제

학생 교안 PDF · 내용 보강본

설치·로그인·장비 준비와 학습 성취는 구분합니다. 이 교안의 배정 시간은 실제 초심자 리허설 결과가 아닙니다. 코드·제공 출력·실물 검증의 한계는 해당 웹 수업과 교안 설명을 따릅니다.